Smile Detection Model Comparison using CNNs, Autoencoders and Classical Methods

Melanie Astrid Montaño Ramos A01643711[1]

[a]Tec de Monterrey, Campus Guadalajara

Este trabajo muestra la implementación de 3 modelos que resuelven el problema de la clasificación binaria de caras sonrientes y no sonrientes; cada uno tiene complejidad diferente, y este proyecto ayuda a demostrar que con las condiciones de escasa data, los modelos más sofisticados no son capaces de funcionar de la mejor manera.

La detección de emociones es un área clave de la visión computacional, ya que permite a las máquinas interpretar señales no verbales presentes en la interacción humana.

Detectar si una persona está sonriendo o no a partir de una imagen. Comparar tres enfoques diferentes:

1. Una red neuronal (CNN)
2. Un autoencoder
3. Un modelo clásico (SVM)

Con este análisis se busca comprobar si existe relación entre la complejidad de un modelo y su eficiencia para segmentar correctamente caras sonrientes y no-sonrientes.

Los datos usados son un subconjunto de *Labeled Faces in the Wild (LFW)* una base de datos de fotografías usadas para estudiar problemas de reconocimiento facial, las cuales fueron recortadas para estar centradas solo en la cara del sujeto al que retratan. Estas representan condiciones reales como desalineación, variación de escala y rotación.

La base cuenta con 1203 imágenes de 64x64 de resolución identificadas previamente como "Smiling" (600 fotos) y "Non-smiling" (603 fotos). En el proceso se identificó que algunas de las fotos estaban erróneamente identificadas (*Figura 1*), y las implicaciones de esto se abordan en el análisis.

Fig. 1. Imágenes de cada clase (primera fila) y otras mal etiquetadas (fila de abajo)

Metodología

A. Carga y procesamiento de las imágenes. Se proporcionaron 2 listas "NON-SMILE_list.txt" y "SMILE_list.txt", y una carpeta con los archivos de las imágenes en formato '.ppm'; dichas listas contienen los nombres de los archivos de las imágenes pertenecientes a cada clase, las cuales se utilizaron para crear el dataset.

Se iteró sobre estas listas para procesar cada imágen convirtiéndola en tensores de *Pytorch* (asociados con su respectiva etiqueta), que se normalizaron con media y desviación estándar de 0.5 para pasar de estar entre [0,1] a [-5,5], lo que mejora el flujo de gradientes.

B. Splitting del dataset. En motivo de entrenar el autoencoder como detector de anomalías, fue necesario entrenar al modelo usando solo una clase, definida como *normal*, así, se estableció a la clase 1 (*Smilling*) como el centro del entrenamiento.

Para el entrenamiento se seleccionaron 400 imágenes de la clase 1 y 200 de la clase 0, pues, para hacer justo este modelado, y evitar pensar que el autoencoder rinde peor solo porque tuvo menos datos, para este se usaron todas estas imágenes de la clase 1, y para los otros dos modelos se usó un subset de 200 imágenes de este, y las 200 imágenes antes apartadas de la clase 0.

Las imágenes restantes se agruparon, remezclaron y dividieron equitativamente para obtener los conjuntos de validación y evaluación usados para los 3 modelos. Resultando entonces conjuntos de 400, 301 y 302 imágenes, para el training, validation y testing, respectivamente.

C. Arquitecturas de los modelos.

C.1. CNN (LeNet-5 adaptado). **Training**

Se implementó una arquitectura inspirada en *LeNet-5 adaptado*, pero utilizando técnicas modernas, esto pues al ser una red poco profunda, se consideró adecuado para el tipo de problema que tenemos (clasificación binaria con pocas muestras de entrenamiento), buscando evitar que el sobreajuste de los datos. Esto y por haber trabajado anteriormente encontrando los hiperparámetros más adecuados para esta arquitectura.

La arquitectura se basa en adaptaciones que buscan preservar la estructura clásica de las convoluciones 5x5 al incorporar mejoras modernas que favorecen la estabilidad del entrenamiento. El modelo inicia con capas convolucionales que extráen las características de la imagen, seguidas de Batch

Normalization y funciones de activación ReLU, que permiten mitigar problemas con los gradientes, y que no existían cuando se propuso por primera vez esta estructura.

Por tratarse de imágenes con diferente resolución a las del problema que resuelve *LeNet-5* (1x28x28), al final de las capas previas a las fully-connected se contaba con una estructura de 120x13x13, a las que se les aplicó un pooling de average progresivo en 3 capas que disminuyeron su resolución en factor de 2, hasta llevar las representaciones hasta un tamaño 1x1. Finalmente, las capas fully connected operan sobre estas representaciones de alta profundidad de la misma forma que el modelo clásico, con profundidades de 120 y 84, y difieren en la salida de 2 para hacer la clasificación.

Para la actualización de pesos se utilizaron los optimizadores Adam y Weight Decay para mejorar el desempeño durante la fase de test. El optimizador Adam es adecuado para datasets pequeños y medianos pues permite ajustar automáticamente la tasa de aprendizaje de los parámtetros; mientras que el uso de Weight Decay, equivalente a regularización L2, contribuye a controlar la magnitud de los pesos del modelo y ayuda a prevenir overfitting, mejorando la capacidad de generalización de la red durante la evaluación.

C.2. Autoencoder (Anomaly Detection). **Training** El modelo fue diseñado para aprender la distribución de una sola clase y detectar anomalías por error de reconstrucción, utilizando una arquitectura encoder–decoder con convoluciones simétricas. El encoder reduce progresivamente la resolución espacial mientras incrementa la profundidad, siguiendo la secuencia ($3\times64\times64 \rightarrow 64\times32\times32 \rightarrow 128\times16\times16$), permitiendo extraer representaciones latentes compactas. Posteriormente, el decoder reconstruye la imagen mediante upsampling (nearest) y convoluciones, invirtiendo el proceso ($128\times16\times16 \rightarrow 64\times32\times32 \rightarrow 3\times64\times64$), un ejemplo del resultado se muestra en la *Figura 2*. Finalmente, se emplea activación Tanh para obtener reconstrucciones en el rango normalizado, lo que facilita la evaluación del error de reconstrucción.

Testing

La evaluación del modelo se realiza a partir de la curva *ROC* (*Figura 3*), donde cada punto de la curva corresponde a un umbral (threshold) distinto sobre el error de reconstrucción. Para clasificar una imagen como "normal" o "anómala", es necesario definir un umbral aplicado a dicho error (pues no es un problema de clasificación en sí), lo que permite analizar la relación entre *True Positive Rate* y *False Positive Rate*. El AUC, o Area Under the Curve, por sus siglas en inglés, proporciona una medida global de la capacidad de segmentación; con un AUC de 0.579 tenemos que el autoencoder no aprendió a separar claramente ambas clases.

D. SVM + HOG. El enfoque *SVM + HOG* se basa en una cadena de procesamiento compuesta por tres etapas: En primer lugar, *HOG (Histogram of Oriented Gradients)* convierte cada imagen en un vector de características, capturando la distribución local de gradientes (*Figura 4*). Posteriormente, se aplicó PCA buscando conservar las variables que explicaran el 95% de la varianza, lo que permitió reducir la dimensionalidad de 1764 a 248 features, facilitando un entrenamiento más eficiente. Finalmente, el algoritmo *Support Vector Machine* busca el hiperplano que maximiza el margen entre clases, utilizando un kernel lineal para interpretabilidad. A propósito se buscó utilizar un modelo simple que contrastara con la complejidad

Fig. 2. Imagen antes y después de ser procesada por el autoencoder

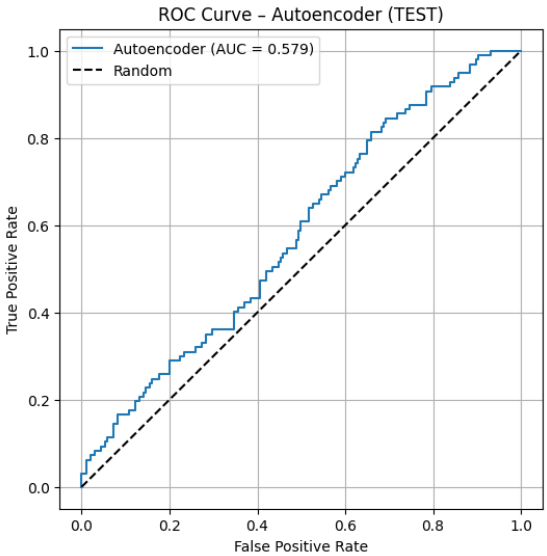

Fig. 3. Curva ROC usada para definir el umbral adecuado para clasificar los errores de reconstrucción

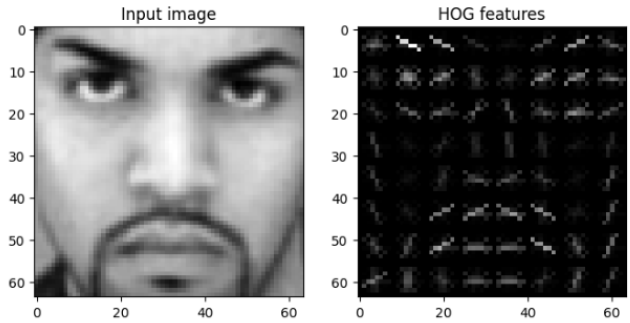

Fig. 4. Sample convertido a grayscale vs. su HOG

Table 1. Accuracy obtenido por modelo

CNN	Autoencoder	SVM
0.8774	0.4901	0.9304

de las redes neuronales, pues de esta manera la comparación rendimiento/complejidad podría ser más contrastante.

1. Resultados

A. Accuracy. De la etapa de testing de cada modelo se crearon arrays con las predicciones individuales de cada modelo, con la intención de evaluar el accuracy, definido como la proporción entre **#imágenes clasificadas correctamente** y el **total de imágenes evaluadas**, es decir, su capacidad de discriminar entre imágenes de las dos clases.

Como se preveía en las curvas *ROC* del autoencoder, el clasificador fue tan bueno como asignar etiquetas aleatoriamente a las imágenes.

Por otro lado, los resultados de la CNN evidencian la presencia de overfitting, o bien, de una capacidad de generalización limitada, que se confirma al analizar la evolución del accuracy, que se vuelve inestable durante la evolución de las épocas, como se aprecia en la *Figura 5*.

En contraste, el accuracy obtenido por el SVM refleja que el uso de HOG, junto con la reducción de dimensionalidades es de gran ayuda para el desempeño del modelo. Como aprecia en las características HOG, este descriptor hace un buen trabajo al extraer los rasgos más relevantes para la detección de una cara sonriente, tal como se muestra en la *Figura 6*.

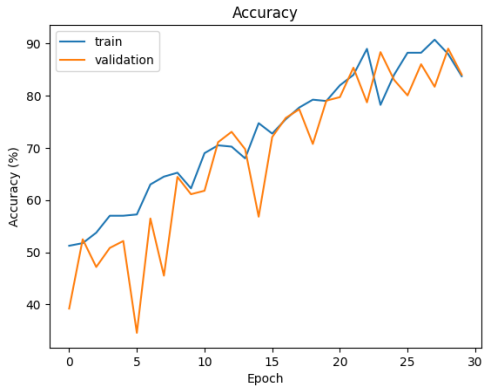

Fig. 5. Accuracy obtenido durante el entrenamiento/validación

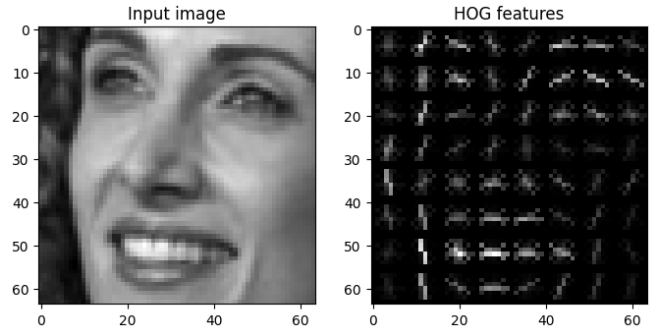

Fig. 6. El HOG es capaz de extraer los rasgos finos que se alteran al sonreir una persona

	CNN	AE	SVM
TIEMPO DE ENTRENAMIENTO	73.594 seg	394.7142 seg	0.0162 seg
TIEMPO DE INFERENCIA	0.827 ± 3.360 ms	0.592 ± 1.046 ms	0.422 ± 0.215 ms
# DE PARÁMETROS	61,610	151,683	249

Fig. 7. Métricas para evaluar la complejidad del modelo

B. Otras métricas.

- Tiempo de entrenamiento: Duración de un algoritmo en analizar los datos de entrada para ajustar los pesos que le permitan minimizar una función de pérdida. A mayor complejidad del modelo (o del volumen de datos), mayor es el tiempo de entrenamiento.

- Tiempo de inferencia: Tiempo que tarda el modelo en dar una respuesta una vez que ya está entrenado. En la práctica el modelo se usa así: una imagen → una decisión → rápido. Se mide como la media y desviación est. de la ejecución de un mismo sample por el forward (predicción) 100 veces.*

- Número de parámetros: Variables aprendidas durante el entrenamiento. En estos modelos son los pesos y sesgos.

*Para el tiempo de inferencia calculado para el SVM se omitió considerar la parte del preprocesamiento de la imágen (HOG + PCA), por lo que pasaría a ser el modelo con mayor tiempo de inferencia requerido

Fig. 8. La boca es la característica espacial más relevante para la clasificación

2. Discusión

- La cantidad de datos fue determinante para que el modelo más simple obtuviera el mejor rendimiento, pues esto le permitió adaptarse mejor a la estructura del dataset.
- La CNN aprende directamente de los pixeles, lo que la hace más flexible pero también más exigente en términos de datos. Aunque la arquitectura está bien diseñada para evitar overfitting, el tamaño del dataset limita la capacidad del modelo para generalizar completamente.
- HOG funciona bien porque las sonrisas se manifiestan como cambios en gradientes, especialmente en la región de la boca. Esto sugiere que la información relevante para detectar sonrisas está principalmente en gradientes y bordes.
- El SVM lineal es suficiente porque las características del espacio HOG ya separan bien las clases.
- El etiquetado ambiguo pudo hacer que el autoencoder se confundiera, además de que gran parte de la imagen (estructura facial) es compartida por ambas clases, y a que la diferencia principal, que es el área de la boca, puede no llegar a representar ni siquiera 1/9 parte del mapa de características (*Figura 8*).

3. Conclusión

En conclusión, los resultados obtenidos muestran que el incremento en la complejidad computacional no garantiza una mejora en el rendimiento, y que la elección del modelo debe estar guiada por la naturaleza del problema y del dataset.

Este proyecto evidencia que entender la naturaleza del problema y del conjunto de datos es tan importante como elegir el modelo, ya que enfoques simples pueden ser altamente efectivos cuando se usan correctamente, logrando un balance adecuado entre desempeño, interpretabilidad y costo computacional.